DELHI TECHNICAL CAMPUS

Affiliated to GGSIPU, New Delhi | NAAC-A++ | Greater Noida

Session 2026–27 • OOPS USING C++

EV Charging Station Management System

C++17 Smart Queue & Full-Stack Demonstration

Introduction

EV charging requires decisions about vehicle range, waiting time, charging deadlines, budget and available power.

This project models these decisions using a modular **C++17 charging engine**, supported by a web interface and simulated charging updates.

The customer app, **Pluggo**, supports vehicle selection, station discovery, charging sessions and demo receipts.

Scope: an educational simulation with no real charger control or live payment.

Objectives

- Apply **encapsulation** through vehicle, user, charger, session and station classes.
- Demonstrate default, parameterized and copy constructors, destructors and function overloading.
- Build a reusable **SmartPriorityQueue<T, ScorePolicy>** with FIFO tie-breaking.
- Plan charging within deadline, connector and budget constraints.
- Simulate solar/grid allocation and emergency preemption with compensation.
- Validate inputs using exceptions and demonstrate behavior through automated checks.

Feasibility Study and Tools Used

Technical: C++17 domain logic with a Crow JSON API, browser UI and SQLite customer storage.

Economic: open-source tools; no dedicated charging hardware needed.

Operational: customer and operator interfaces support a classroom demonstration.

Core tools: C++17, CMake, CTest, Crow/Asio, HTML/CSS/JavaScript and SQLite.

Supporting components: Leaflet/OpenStreetMap, Python charging simulator, Flutter app and Docker.

System Architecture

CLIENT LAYER

Pluggo web app / Flutter app

↓

API LAYER

Crow REST / JSON endpoints

↓

C++ DOMAIN LAYER

Queue • pricing • charging rules

Storage: SQLite persists customer vehicles, sessions and receipts. Legacy operator sessions remain in memory.

Simulation: WebSocket / OCPP simulator supplies charging updates through the API boundary.

Design: Components and Concepts Used

Component	OOP / C++ concept
Vehicle, User, Charger, Session, Station	Encapsulation; constructors and destructors
SmartPriorityQueue<T, ScorePolicy>	Class template; score ordering; FIFO ties
PricingEngine:: calculateCost	Overloading: flat, time-of-use and solar pricing
logEvent<T>	Function template
Input validation	Exception handling

Implementation

Simulated charging workflow

1. Select a vehicle and compatible station.

2. Validate range, deadline and budget.

3. Queue the request; allocate a compatible charger by priority.

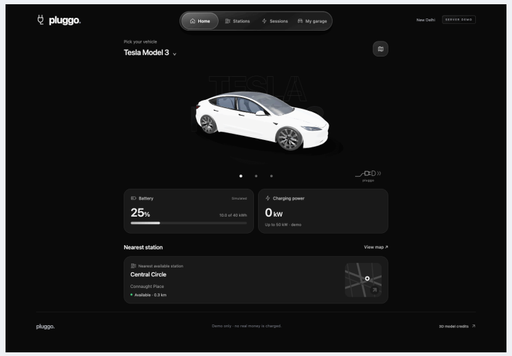
4. Simulate charging using solar power with grid fallback.

5. Show battery state, energy, cost and session history.

Emergency handling: pause an eligible normal session and compensate the affected user.

Additional features: charger QR scanning, an India station directory, map search and directions.

Result and Demonstration



Actual Pluggo customer interface • simulated data

Automated check coverage: range boundaries; deadline feasibility; connector compatibility; pricing and budget caps; queue ordering and FIFO ties; solar fallback; emergency preemption and rejection.

Conclusion

C++ classes, templates and overloads connect charging decisions to an interactive application.

Future scope: PostgreSQL integration, Keycloak JWT validation and an audited OCPP service. Current authentication and charger interaction are for demonstration.

Submission by

BHUMIKA (00818015625)
ADITI RAJ (11318015625)
RONIT SHARMA (01018015625)
SARTHAK KUMAR (05318015625)
CSE-AIML | Submitted to: Ms. Kajal Kumari

Scan: source code